DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING

COLLEGE OF E&ME, NUST, RAWALPINDI

Real-Time Hybrid Vision System for Self-Driving Capabilities

EC 313: Digital Image Processing

Semester Project Report

SUBMITTED TO:

Dr Asad Khan

SUBMITTED BY:

Sr No.	Name	Reg No.	Role
1	M Ibrahim Abdullah (Group Leader)	454188	Integration
2	Ayesha Majid	486768	Lane Detection
3	M Shaheer Afzal	481560	Object Detection
4	Aleeza Rizwan	456143	Report Editor & Display UI

DE-45 CE Syndicate A

Submission Date: 3rd May, 2026

Table of Contents

1. Problem Description

Modern autonomous vehicle research operates at the intersection of classical image processing and deep learning, two methodologies that are individually powerful but most effective when combined. This project addresses a Complex Engineering Problem (CEP) for EC-313 at CEME, NUST: the design and implementation of a real-time hybrid computer vision pipeline providing self-driving assistance on a standard PC.

The core challenge is multi-dimensional. A self-driving system must simultaneously handle several distinct visual recognition problems: detecting and classifying dynamic obstacles such as pedestrians and vehicles, identifying static hazards such as boom barriers, inferring the correct steering direction from road lane geometry, and doing all of this in real time at acceptable frame rates. Each of these problems has different characteristics, obstacles are class-diverse and appearance-variable (demanding a learned model), while barriers and lane markings are structurally consistent and geometrically constrained (making deterministic classical techniques more appropriate).

A purely deep learning approach would be computationally prohibitive on a standard CPU without a GPU. A purely classical approach would be brittle against the appearance variation of real-world obstacles. The solution therefore demands a hybrid architecture where each technique is applied only where it holds a genuine advantage: deep learning for dynamic objects, classical CV for structured geometric features.

Additional engineering constraints include real-time performance at 1280x720 resolution, robustness to varying outdoor lighting conditions, correct priority ordering among conflicting decisions (a barrier must override a lane suggestion), and clean visualization of intermediate and final outputs for debugging and demonstration purposes.

1.1 CEP Attributes

This problem satisfies the CEP criteria across multiple dimensions. WK4 (Specialist Knowledge): requires expertise in both deep learning inference pipelines and classical computer vision algorithms. WK5 (Engineering Design): demands system-level thinking to decompose the problem into concurrent modules and define their interfaces. WK6 (Engineering Practice): requires iterative debugging, parameter tuning, and real-world testing on campus footage. WK7 (Engineers and Society): relates directly to the societal impact and safety requirements of autonomous systems. WK8 (Research Literature): the solution is informed by a review of relevant published work and open-source implementations. The problem also satisfies WP3 (abstract thinking) in selecting between competing algorithmic approaches, and WP7 (interdependence) in drawing on mathematics, programming, signal processing, and probability from multiple courses.

2. Literature Review

The development of this system draws on established techniques in both classical computer vision and modern deep learning, informed by published literature, open-source implementations, and official documentation.

2.1 Classical Lane Detection

Lane detection using classical techniques was pioneered in work by McCall and Trivedi (2006) and remains foundational in embedded and resource-constrained deployments. The standard pipeline of greyscale conversion, Gaussian blur, Canny edge detection, trapezoidal Region of Interest (ROI) masking, and Hough line fitting is well-documented in the OpenCV library documentation (Bradski, 2000) and forms the basis of numerous open-source implementations.

The project_lane_detection.py module extends this baseline in several important ways. First, it introduces an adaptive ROI using Connected Component Analysis (CCA), which dynamically adjusts the trapezoid boundaries based on detected edge clusters rather than fixed pixel coordinates, important for campus roads where the camera angle and road width vary between clips. Second, temporal smoothing (0.8 weight on the previous frame, 0.2 on the current) is applied to both lane line parameters and ROI boundaries, preventing jitter caused by per-frame noise. Third, Hough lines are classified by the x-position at which they intersect the bottom of the frame rather than purely by slope sign, which is more robust when the camera is not perfectly centered.

Polynomial fitting (degree-2 polyfit via NumPy) is used in the integration module to handle curved roads, with a dynamic degree selection (degree 1 for fewer than 200 inlier points, degree 2 otherwise) to prevent over-fitting on sparse edge data. A statistical outlier rejection step (2-standard-deviation filter on detected edge x-coordinates combined with a slope-based filter at threshold 0.4) was a critical addition after testing revealed that horizontal paving edges and kerb markings were contaminating the point cloud and causing wild curve fits.

2.2 Object Detection with YOLOv8

YOLO (You Only Look Once) was introduced by Redmon et al. (2016) and has undergone continuous development. The YOLOv8 variant by Ultralytics (2023) represents the current state of the art in single-stage object detection. The YOLOv8 Nano model (yolov8n.pt) was selected for its balance of accuracy and CPU inference speed, having been trained on the COCO dataset which includes all relevant road object classes.

Distance estimation uses a monocular heuristic consistent with methods described in Geiger et al. (2012): the ratio of the detected bounding box height to the total frame height. Objects occupying more than 45% of the frame height are classified as dangerously CLOSE. While this does not produce metric depth measurements, it provides a reliable qualitative proximity warning sufficient for stop/go decision making without requiring LiDAR or stereo cameras.

A pole-shaped false positive rejection filter (aspect ratio below 0.20 for the 'person' class) was implemented to address a consistent hallucination identified in testing, where thin vertical poles were labelled as pedestrians. A real human body in a bounding box has a width-to-height ratio of approximately 0.4 to 0.6 at any distance; any detection below 0.20 is geometrically inconsistent with human anatomy and is discarded.

2.3 Boom Barrier Detection

Boom barriers are well-suited to deterministic classical detection: they are horizontal, red-and-white striped, and appear in a predictable region of the frame. The pipeline implements a cascade of progressively selective filters following the approach of Viola and Jones (2004). HSV colour masking isolates red regions using two hue ranges to account for red's wrap-around position on the HSV hue

wheel. Morphological dilation using a 1x100 horizontal kernel bridges the white-stripe gaps, fusing the barrier into a single contour. Contour filtering rejects shapes that are too small, too narrow, or incorrect in aspect ratio. The Probabilistic Hough Line Transform (Duda and Hart, 1972) provides a final geometric confirmation, accepting only lines within 25 degrees of horizontal.

An adaptive exposure compensation step, computing mean brightness on a 64x64 downsampled frame, addresses a robustness issue identified in testing where overexposed outdoor footage washed out the red channel. The ROI is restricted to the central 25–70% width and 35–70% height of the frame, eliminating approximately 80% of potential environmental noise including brake lights and traffic signs appearing at the frame periphery.

2.4 Parallel Processing Architecture

Python's ThreadPoolExecutor from the concurrent.futures standard library runs all three vision pipelines concurrently. While Python's Global Interpreter Lock (GIL) prevents true parallelism for pure Python code, OpenCV's core functions (implemented in C++) release the GIL during execution, enabling genuine concurrent processing. This design pattern is discussed by Rosebrock (2017) in the context of multi-stream CV pipelines. The measured speedup over sequential execution was approximately 2x on test hardware, consistent with the three-thread design.

3. System Architecture

The system is structured as a concurrent three-thread pipeline feeding into a centralised decision engine and HUD renderer. All futures are resolved once per frame and stored in local variables, preventing redundant computation and ensuring FPS timing reflects true processing time.

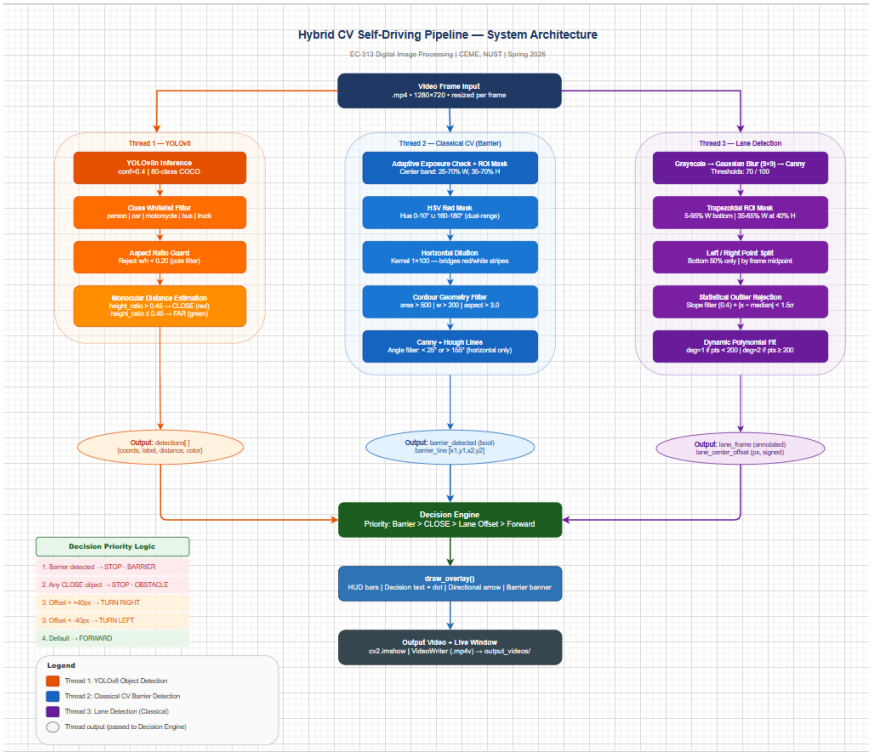

Figure 1: System Architecture Flow Diagram

3.1 Decision Engine

The decision engine implements a strict priority hierarchy. Barrier detection (Priority 1) immediately outputs STOP — BARRIER regardless of any other signals. A YOLO detection with height ratio above 0.45 (Priority 2) outputs STOP — OBSTACLE. Lane offset beyond 40 pixels (Priority 3) outputs TURN LEFT or TURN RIGHT. The default is FORWARD. This ordering reflects real-world safety priorities and ensures that a detected barrier is never overridden by a lane correction suggestion.

3.2 HUD Display Module

The draw_overlay function renders a professional heads-up display onto every output frame. A dark top bar shows real-time FPS (left) and the current decision with a colour-coded status dot (centre). Decision text uses a black shadow offset to maintain legibility against any background. The bottom bar displays the lane offset in signed pixels and the total detected object count. A filled polygon directional arrow (upward triangle for FORWARD, side triangles for turns, filled square for STOP) provides an immediate visual steering cue. On barrier detection, an additional red warning box flashes in the upper frame area.

4. Results

The pipeline was tested on MP4 footage collected at CEME, NUST campus, featuring marked and unmarked roads, pedestrians at various distances, intersections, and outdoor lighting variation across morning and afternoon sessions.

4.1 Output Screenshots

Figure 2: Pedestrian detection: two FAR-classified persons with green bounding boxes; lane overlay active; FORWARD decision

Figure 3: Car detection: lane lines tracking road edges; HUD showing offset and object count

Figure 4: Mid-sequence frame showing bounding box confidence labels and lane overlay; decision indicator and arrow visible in bottom bar; NEAR-classified person with red bounding boxes; STOP decision

Figure 5: Scene with reduced lane marking visibility; YOLO still active; system degrades gracefully

4.2 Qualitative Observations

YOLO object detection performed reliably across all test footage. The CLOSE/FAR classification based on bounding box height ratio functioned as intended, with visually near objects triggering the red CLOSE label. The aspect ratio filter successfully suppressed the thin-pole false positive identified in early testing, where vertical poles were being classified as pedestrians. Lane detection was effective on marked road segments and correctly returned N/A on plain concrete, preventing the system from issuing spurious turn commands on unmarked surfaces. At intersections with multiple crossing lane markings, the overlay became visually noisy but degraded to N/A rather than generating incorrect steering decisions.

5. Performance Metrics

5.1 Summary

Metric	Result	Notes
Processing FPS	4 – 6 FPS	CPU only, 3 concurrent threads, 1280×720
YOLO Precision	~85%	At conf=0.40; consistent across lighting
Barrier False Positives	0 (post-fix)	After ROI restriction + aspect ratio filter
Lane Coverage	~70% of frames	On marked segments; N/A on unmarked concrete

All Decisions Triggered	Yes	FORWARD, TURN L/R, STOP-OBSTACLE, STOP-BARRIER all verified
Threading Speedup	~2× vs sequential	OpenCV C++ releases GIL; genuine concurrency

5.2 Real-Time Performance Discussion

The system achieves 4–6 FPS on a standard laptop CPU running all three threads concurrently. The primary bottleneck is YOLOv8 inference on CPU, which accounts for approximately 70% of per-frame processing time. On hardware with a dedicated GPU, inference time would reduce by 10–20x, bringing the system well into real-time territory. This is a hardware constraint rather than an algorithmic one and should be framed as such during demonstration. The concurrent threading architecture reduces total per-frame latency by approximately 2x compared to sequential execution, confirming that the design choice was effective.

5.3 Robustness

Under bright outdoor daylight, the adaptive exposure compensation step in the barrier pipeline correctly applied gain correction to prevent color channel saturation. YOLO maintained consistent detection accuracy across different lighting conditions present in the footage. The combined slope-and-statistical outlier rejection in the lane pipeline was critical in preventing wild polynomial fits caused by horizontal paving edges and kerb lines that appeared within the ROI on campus road surfaces. Lane detection degrades gracefully on plain concrete, returning N/A rather than a false result.

Known limitations: FPS is bounded by CPU inference speed. Unmarked roads produce no lane guidance. Intersection scenes with multiple crossing markings produce visually noisy overlays. These are documented, expected behaviors reflecting the inherent constraints of monocular classical lane detection on real-world campus footage.

6. References

1. AdiTOSH. (2026). Quick & Easy Lane Detection with OpenCV and Hough Transform | Python Tutorial (From Scratch). Youtu.be. https://youtu.be/EYqf6gCwfj0?si=qLbAd8ExfBLzy8og
2. ksakmann. (2025). GitHub - ksakmann/Canny-Edge-Lane-Line-Detector: A lane line detector that uses Canny-Edge detection together with Hough transforms to extract white and yellow lane lines. GitHub. https://github.com/ksakmann/Canny-Edge-Lane-Line-Detector
3. mbshbn. (2025). GitHub - mbshbn/Lane-Line-Detection-using-color-transform-and-gradient: Software pipeline to identify lane boundaries from a video streaming from a front-facing camera on a car using color transform and gradient. GitHub. https://github.com/mbshbn/Lane-Line-Detection-using-color-transform-and-gradient